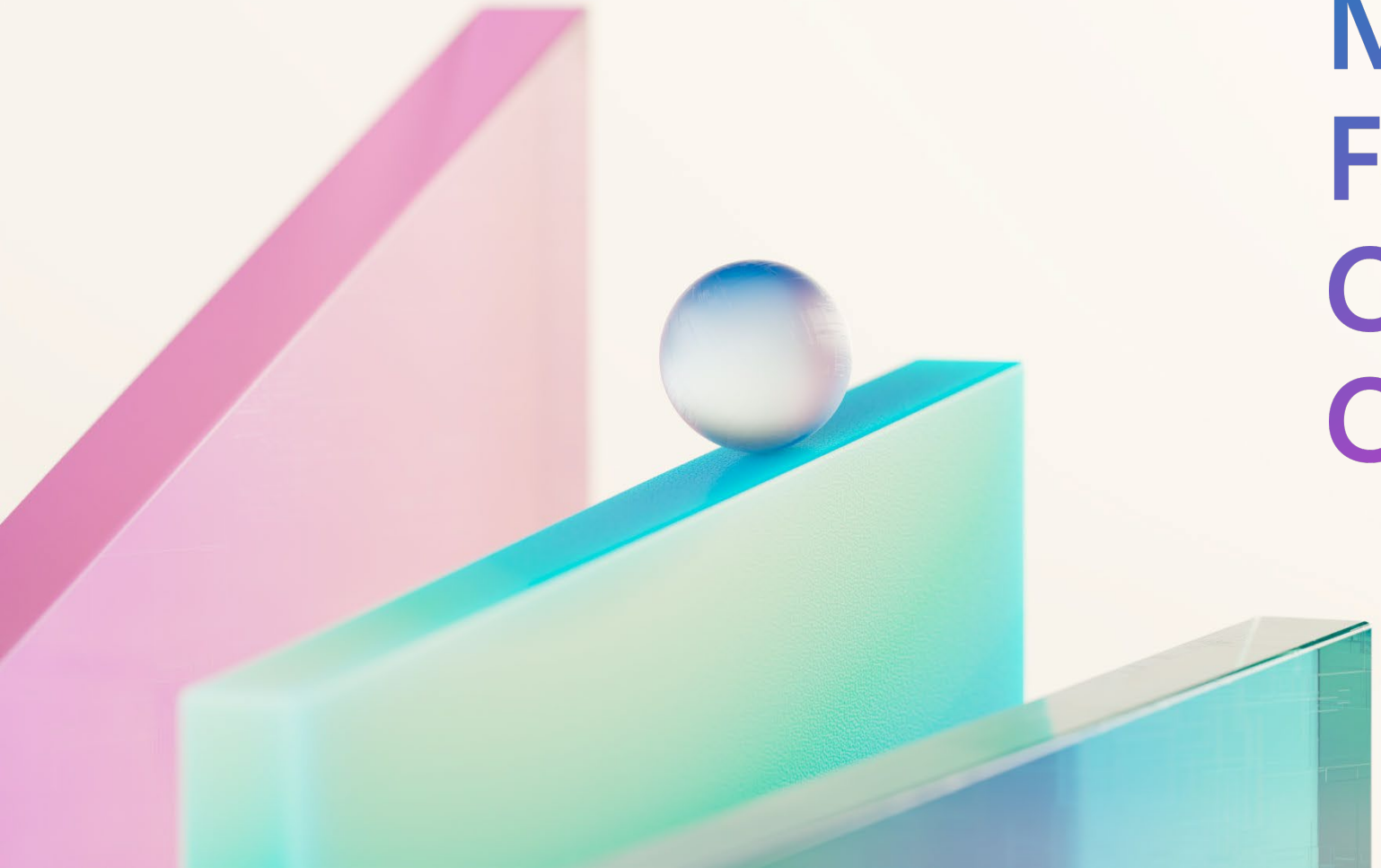
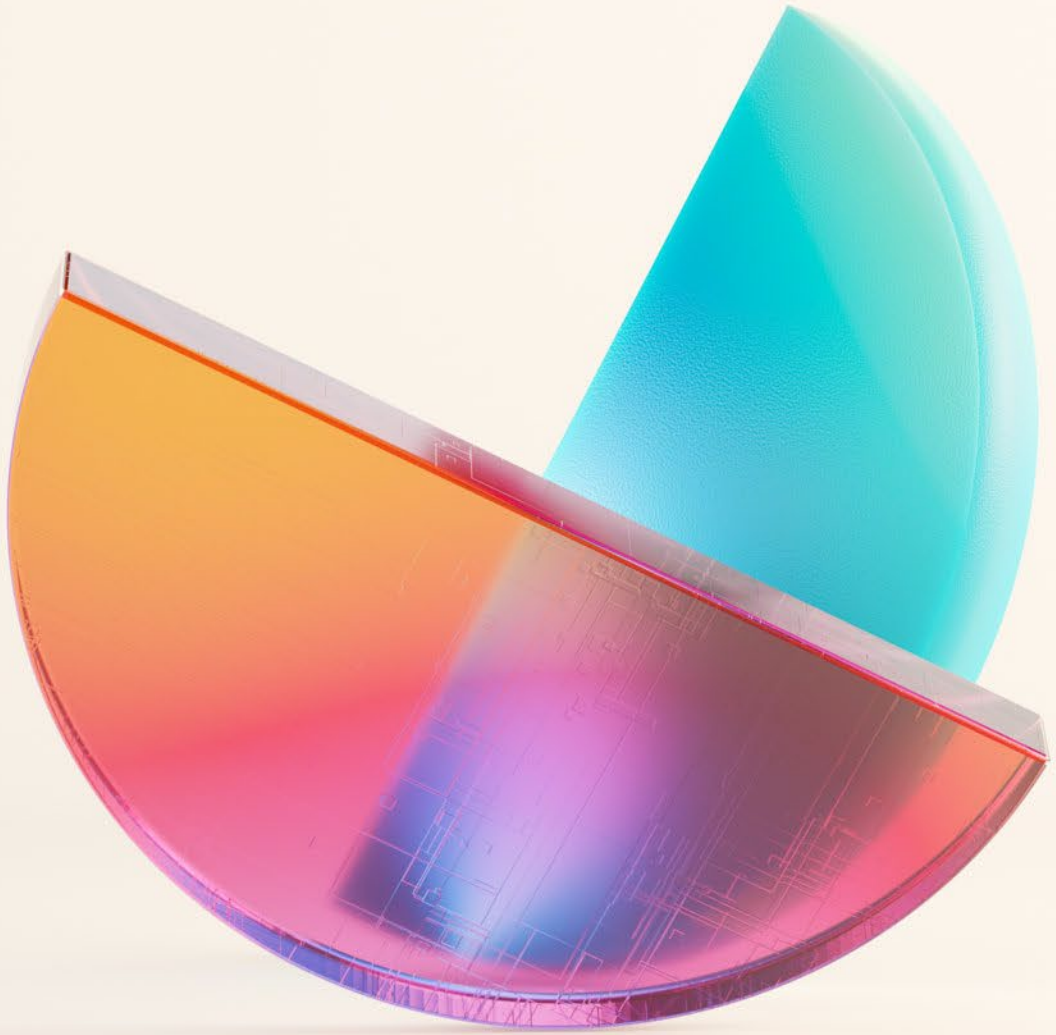
Microsoft



# Microsoft Fabric Community Conference

Microsoft

Microsoft Fabric
Community Conference



# Mastering Lakehouse Maintenance

From Basics to Automation with Semantic Link Labs

Speaker: Roger Unholz

# Agenda

01
Introduction

02
Lakehouse basics

03
Demo - Basics

04
Maintenance basics

05
Demo - Maintenance

06
Semantic Link Labs

07
Demo – Semantic Link Labs

08
Q&A



# ROGER UNHOLZ



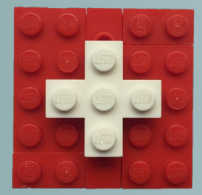
# ROGER UNHOLZ

ZÜRICH, SWITZERLAND



# ROGER UNHOLZ

ZÜRICH, SWITZERLAND



BSGROUP
DATA ANALYTICS



# ROGER UNHOLZ

ZÜRICH, SWITZERLAND

BSGROUP
DATA ANALYTICS

PARTNER &
SOLUTION ARCHITECT



# ROGER UNHOLZ

ZÜRICH, SWITZERLAND

BSG GROUP
DATA ANALYTICS

PARTNER &
SOLUTION ARCHITECT

data conference
SWITZERLAND



# ROGER UNHOLZ


roger.unholz@bsgroup.ch

/in/rogerunholz


ZÜRICH, SWITZERLAND

BSGROUP
DATA ANALYTICS

PARTNER &
SOLUTION ARCHITECT

data conference
SWITZERLAND

# WE ALL WOULD LIKE TO LIVE IN THIS LAKEHOUSE...RIGHT?



# WHAT ABOUT NOW...



# FABRIC LAKEHOUSE ARCHITECTURE

COMPUTE

Lakehouse

Apache Spark

STORAGE

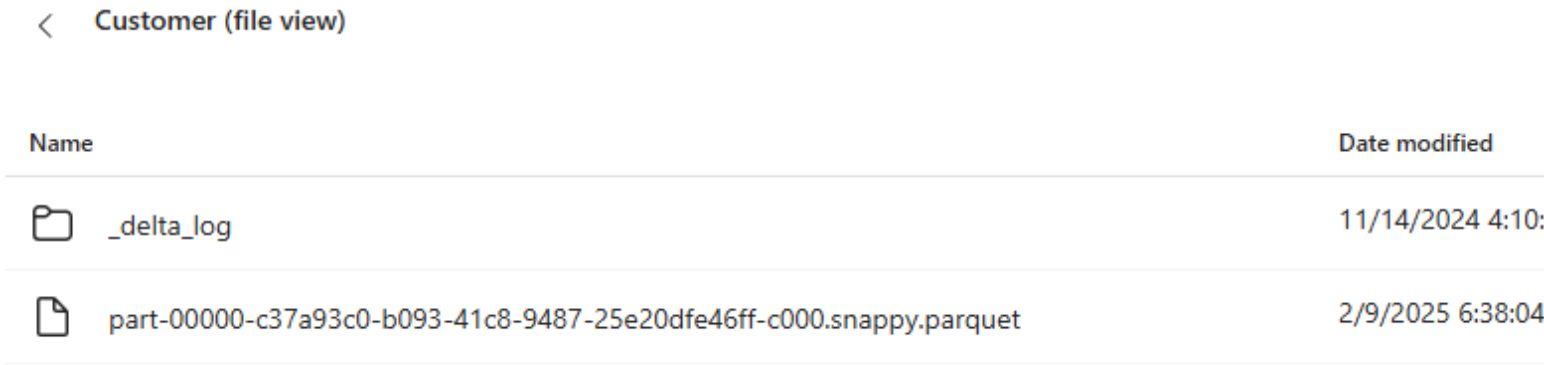
OneLake

DELTA LAKE

DELTA LAKE

- Open-source (originally developed by Databricks)
- ACID transactions
- Metadata handling
- Time Travel

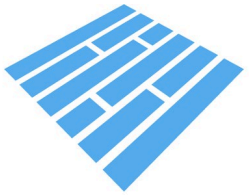
Parquet






# LIFECYCLE OF A DELTA TABLE

- update
- insert
- delete




files are **immutable**

Microsoft Fabric
Community Conference



# Demo - Basics



SalesDetail (file view) > **YearValue=2024**

| Name | Date modified | Type | Size |
|---|---|---|---|
| part-00000-05450e96-98f3-44e5-9f89-3e24461cbe38.c000.snappy.parquet | 2/17/2025 6:44:54 PM | parquet | 226 KB |
| part-00000-06f8281b-94b2-4529-9075-0f4d8c192f67.c000.snappy.parquet | 2/17/2025 6:55:41 PM | parquet | 218 KB |
| part-00000-093c23c6-17e6-4aa1-8d69-6248934fee8c.c000.snappy.parquet | 2/17/2025 6:40:06 PM | parquet | 235 KB |
| part-00000-0b1cf258-2126-4378-9423-e952e027a6d9.c000.snappy.parquet | 2/17/2025 6:40:49 PM | parquet | 206 KB |
| part-00000-1d8acb31-1b3a-459a-bf29-62b1e9c659aa.c000.snappy.parquet | 2/17/2025 6:33:46 PM | parquet | 223 KB |
| part-00000-1f118a3b-d257-4ca7-8868-1416ea2e9aa8.c000.snappy.parquet | 2/17/2025 6:47:05 PM | parquet | 204 KB |
| part-00000-20ee54cf-d67a-4b5f-b138-af16ce9a6187.c000.snappy.parquet | 2/17/2025 6:51:28 PM | parquet | 208 KB |
| part-00000-23bf3fc8-2da7-4dfa-9294-0b8205a77388.c000.snappy.parquet | 2/17/2025 6:39:36 PM | parquet | 252 KB |
| part-00000-23c9c582-76df-4edb-9dcc-3d6c54c6943c.c000.snappy.parquet | 2/17/2025 6:56:47 PM | parquet | 221 KB |
| part-00000-24360aef-2e63-46a0-bf09-f7a0e97b91b2.c000.snappy.parquet | 3/12/2025 9:28:28 PM | parquet | 34 MB |

# SMALL FILE PROBLEM



# MAINTENANCE

## OPTIMIZE

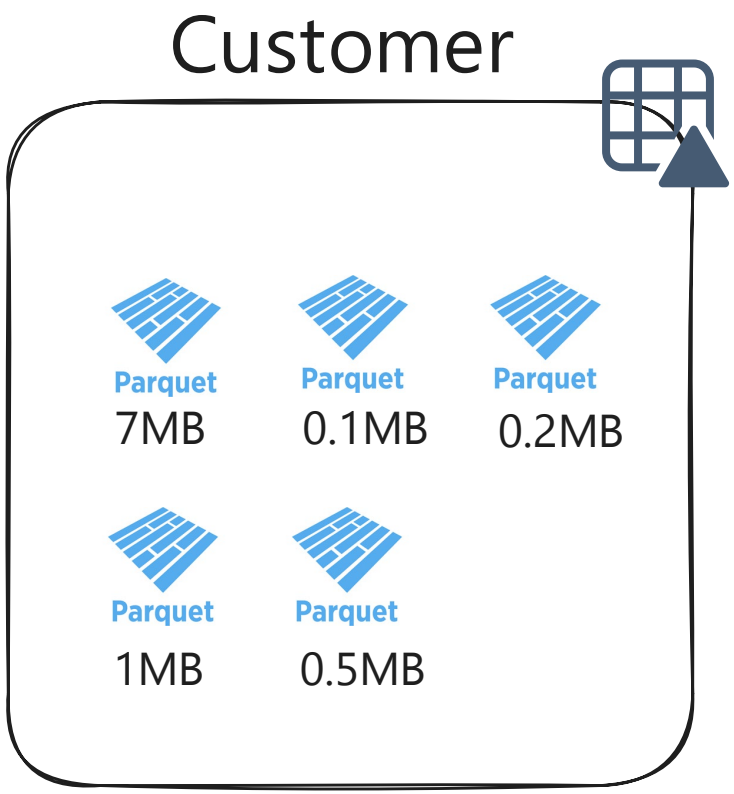
File compactation

Reduze # of small files

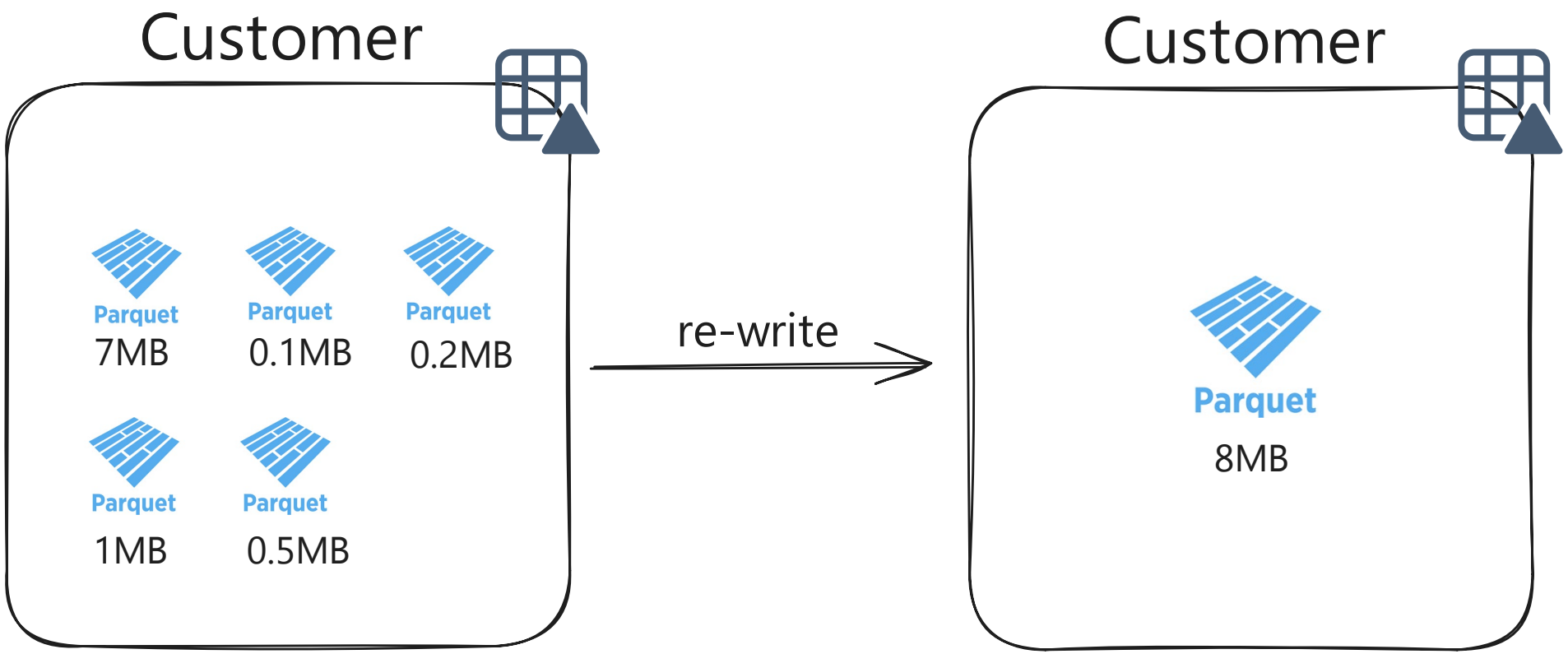
## VACUUM

Removing of un-referenced files

# OPTIMIZE


Customer
Parquet
7MB
Parquet
0.1MB
Parquet
0.2MB
Parquet
1MB
Parquet
0.5MB

# OPTIMIZE


Customer
Parquet
7MB
Parquet
0.1MB
Parquet
0.2MB
Parquet
1MB
Parquet
0.5MB
re-write
Customer
Parquet
8MB

Microsoft Fabric
Community Conference



# Demo - Optimize



# MAINTENANCE

## OPTIMIZE

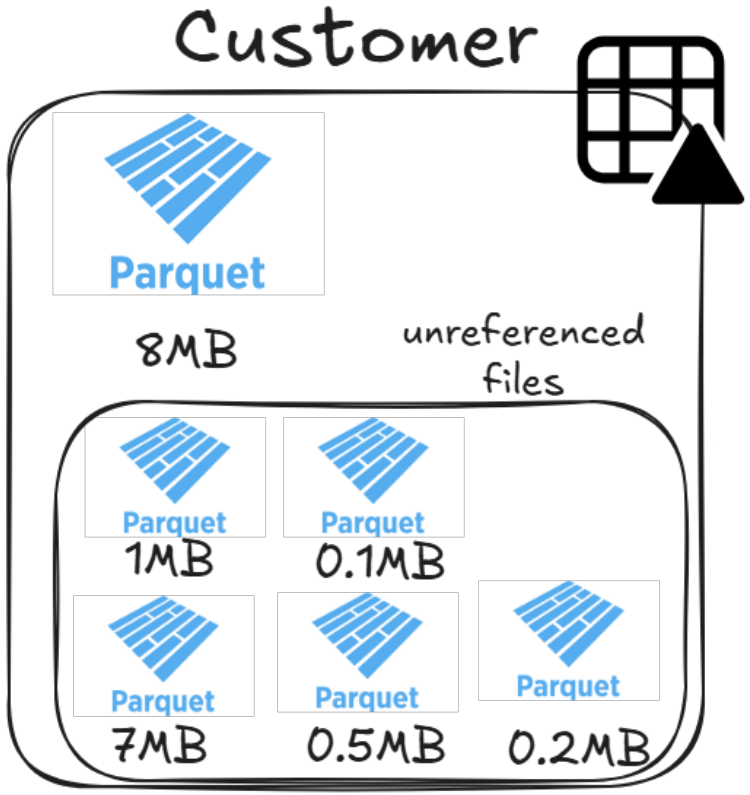
File compactation

Reduze # of small files

## VACUUM

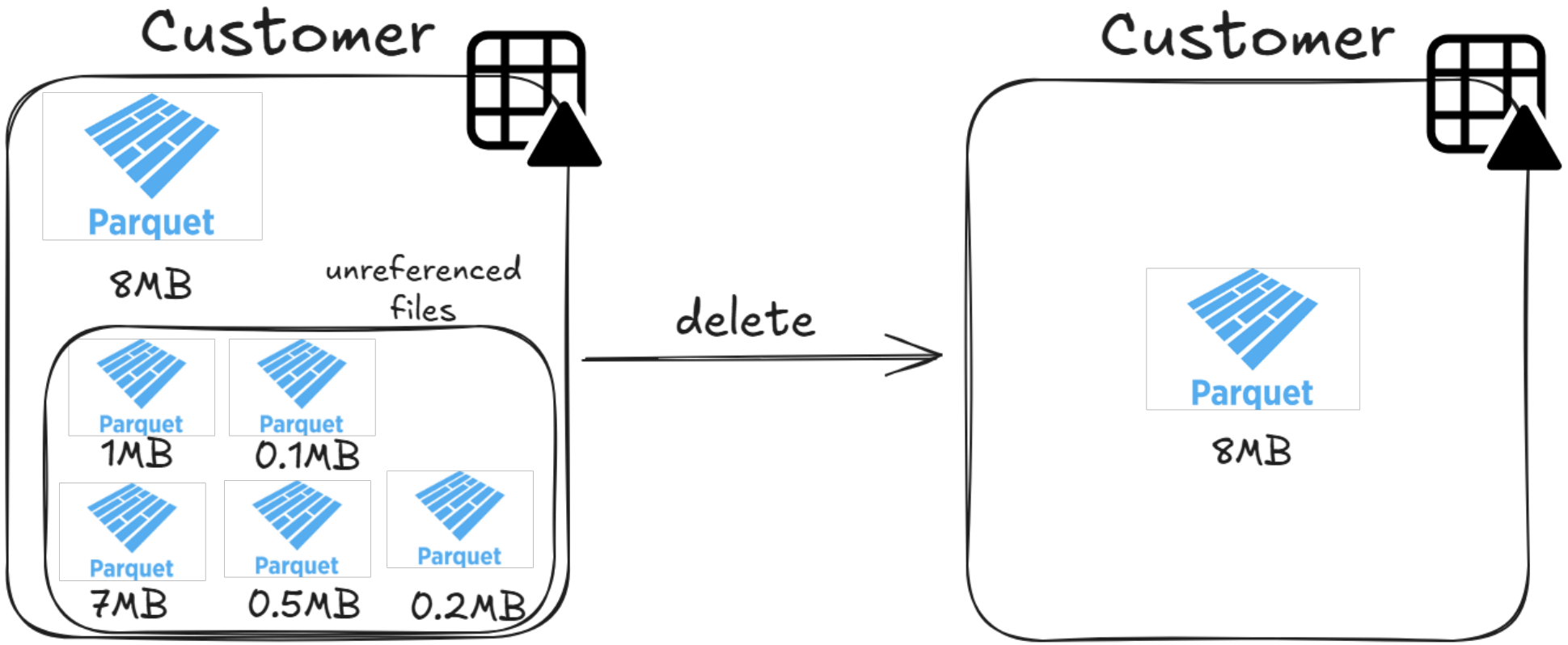
Removing of un-referenced files

# VACUUM


Customer
Parquet
8MB
unreferenced
files
Parquet
1MB
Parquet
0.1MB
Parquet
7MB
Parquet
0.5MB
Parquet
0.2MB



# VACUUM


Customer
Parquet
8MB
unreferenced files
Parquet
1MB
Parquet
0.1MB
Parquet
7MB
Parquet
0.5MB
Parquet
0.2MB
delete
Customer
Parquet
8MB

Microsoft Fabric
Community Conference

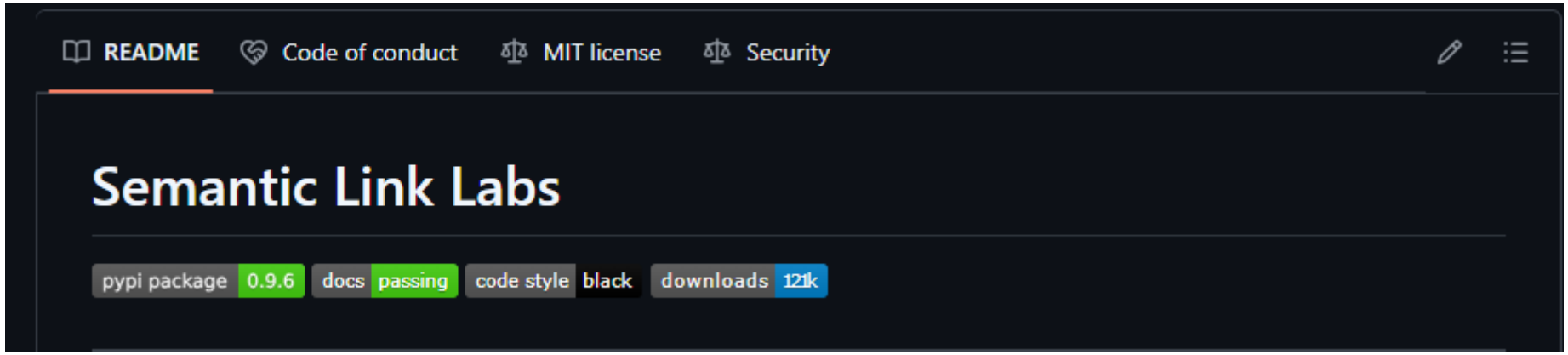


# Demo - Vacuum

# SEMANTIC LINK LABS

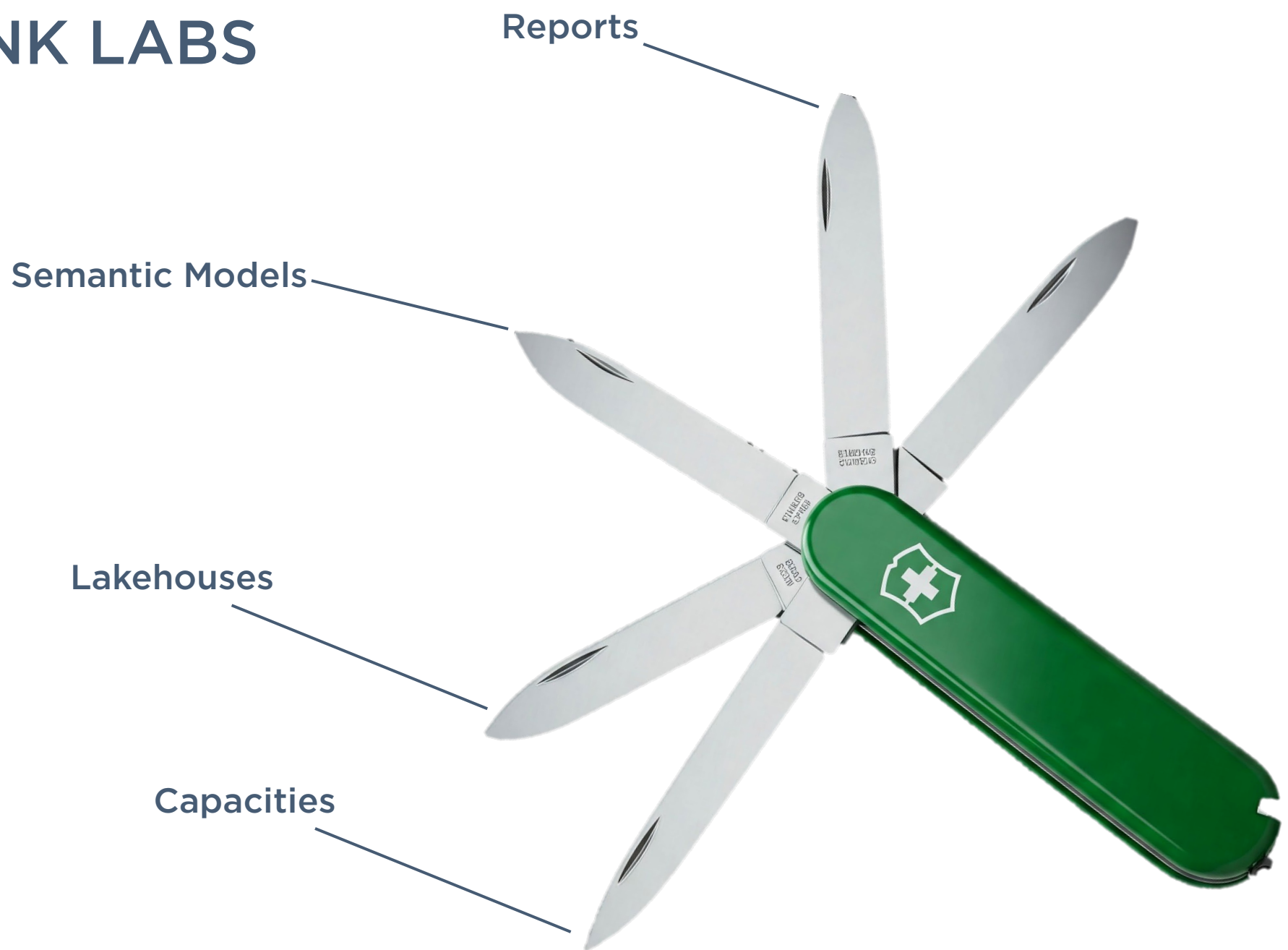
A Python library for Fabric notebooks to simplify technical tasks

https://github.com/microsoft/semantic-link-labs



# SEMANTIC LINK LABS





# SEMANTIC LINK LABS



Reports

Semantic Models

Optimize Lakehouse tables

Lakehouses

Vacuum Lakehouse tables

Capacities



Microsoft Fabric
Community Conference



# Demo – Semantic Link Labs

# LAKEHOUSE MAINTENANCE – WRAP UP

## 1 We retrieve all tables from our Lakehoues (or multiple Lakehouses)

```
1   df_test = lake.get_lakehouse_tables()
2   display(df_test)
3
```

[3] ✓ - Command executed in 3 sec 770 ms by Roger Unholz | BSgroup Data Analytics on 3:51:31 PM, 3/31/25 PySpark (Python)

Table | New chart | 6 columns, 11 rows

Table view | Download | Search

| | Workspace Name | Lakehouse Name | Table Name | Format | Type | Location |
|---|---|---|---|---|---|---|
| 1 | My workspace | LH_New_RetailStore | AgeGenderBuy... | delta | Managed | abfss://12def... |
| 2 | My workspace | LH_New_RetailStore | CategorySchedule | delta | Managed | abfss://12def... |
| 3 | My workspace | LH_New_RetailStore | TopCategory | delta | Managed | abfss://12def... |
| 4 | My workspace | LH_New_RetailStore | FactualProduct | delta | Managed | abfss://12def... |

Inspect

# LAKEHOUSE MAINTENANCE – WRAP UP

## 2 Create a function that helps us analyze our tables

```
def analyze_delta_maintenance(table_paths):
    """
    Analyzes Delta tables for maintenance needs by checking:
    1. Number of files per partition
    2. File sizes
    3. Last OPTIMIZE operation
    4. Last VACUUM operation

    Parameters:
    table_paths: List of paths to Delta tables

    Returns:
    pandas.DataFrame: Analysis results for each table
    """
```

# LAKEHOUSE MAINTENANCE – WRAP UP

## 3 Check all our delta tables if they need to be maintained

Let's check our Lakehouse!

```
# read all tables
df = lake.get_lakehouse_tables()

#only include tables that are delta format
df_filtered = df[df['Format'] == 'delta']

# display(df_filtered)

analyzed_tables = analyze_delta_maintenance(df_filtered)

display(analyzed_tables)
```

✓ - Command executed in 54 sec 622 ms by Roger Unholz | BSgroup Data Analytics on 3:56:59 PM, 3/31/25

PySpark (Python)

# LAKEHOUSE MAINTENANCE – WRAP UP

## 4 Run optimize / vacuum for all the tables that got flagged

Let's optimize if needed

```
1   table_list = []
2
3   for row in analyzed_tables.rdd.collect():
4       if row.needs_optimize:
5           table_name = row.table_name
6           table_list.append(table_name)
7
8   # Optimize all tables that need optimization
9   if table_list:
10      lake.optimize_lakehouse_tables(tables=table_list)
```

✓ - Command executed in 26 sec 669 ms by Roger Unholz | BSgroup Data Analytics on 2:02:27 PM, 3/27/25 PySpark (Python)

# RECOMMENDATIONS

1. Run optimize regularly e.g. weekly
2. Run vacuum depending on your desired file retention (e.g. 30 days)
3. Optimal file size 100MB to 1GB
4. Auto compacting

**Table property:**

```
ALTER TABLE Customer SET
TBLPROPERTIES
('delta.autoOptimize.autoCompact' = 'true')
```

# RESOURCES

Sandeep Pawar, MVP

https://fabric.guru/

Miles Cole

https://milescole.dev/

# Q & A

# GITHUB REPO

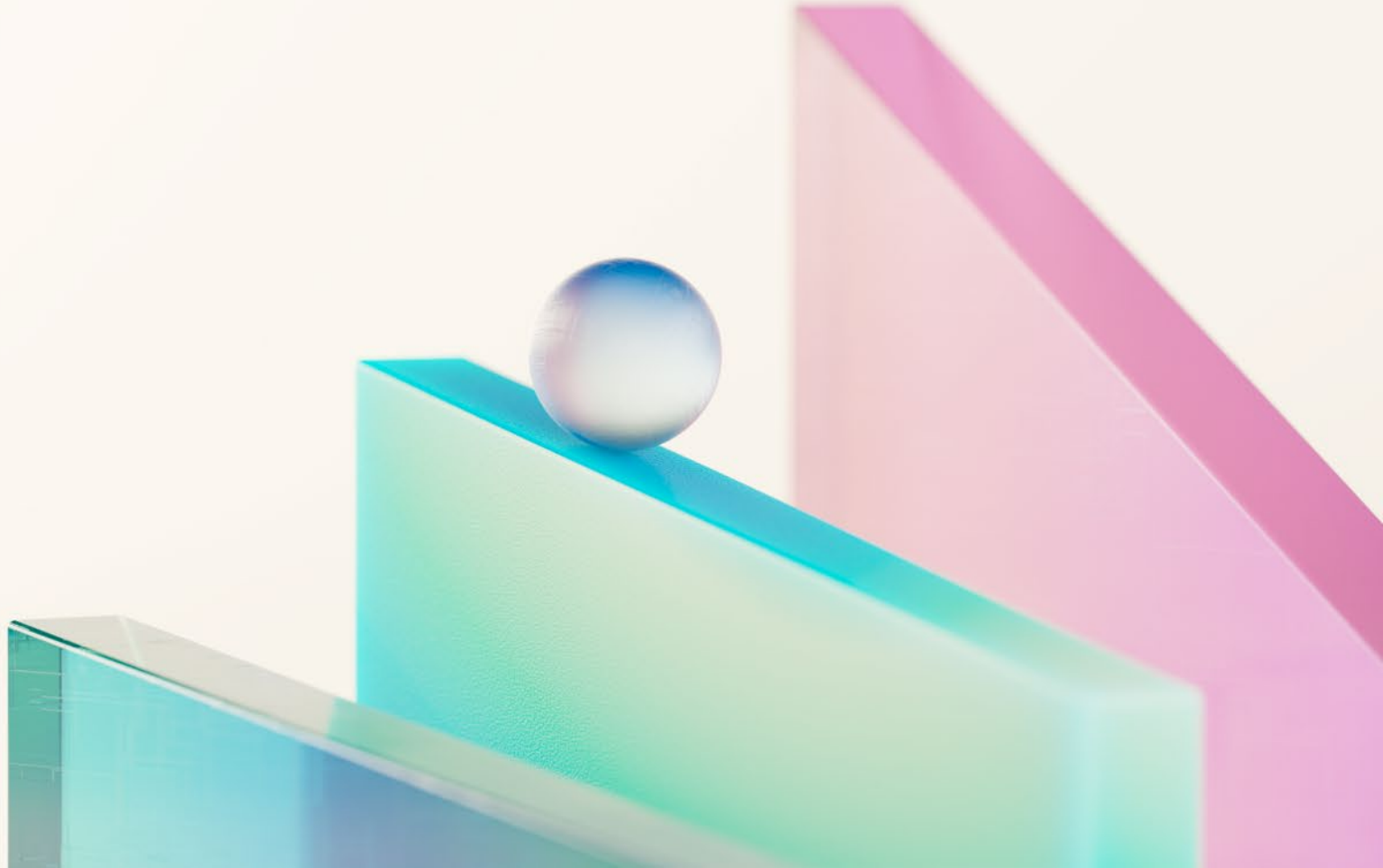


https://github.com/runholz/LakehouseMaintenance

Microsoft

Microsoft Fabric
Community Conference

# THANK YOU



Microsoft Fabric
Community Conference



# Fast-track your career in data and AI

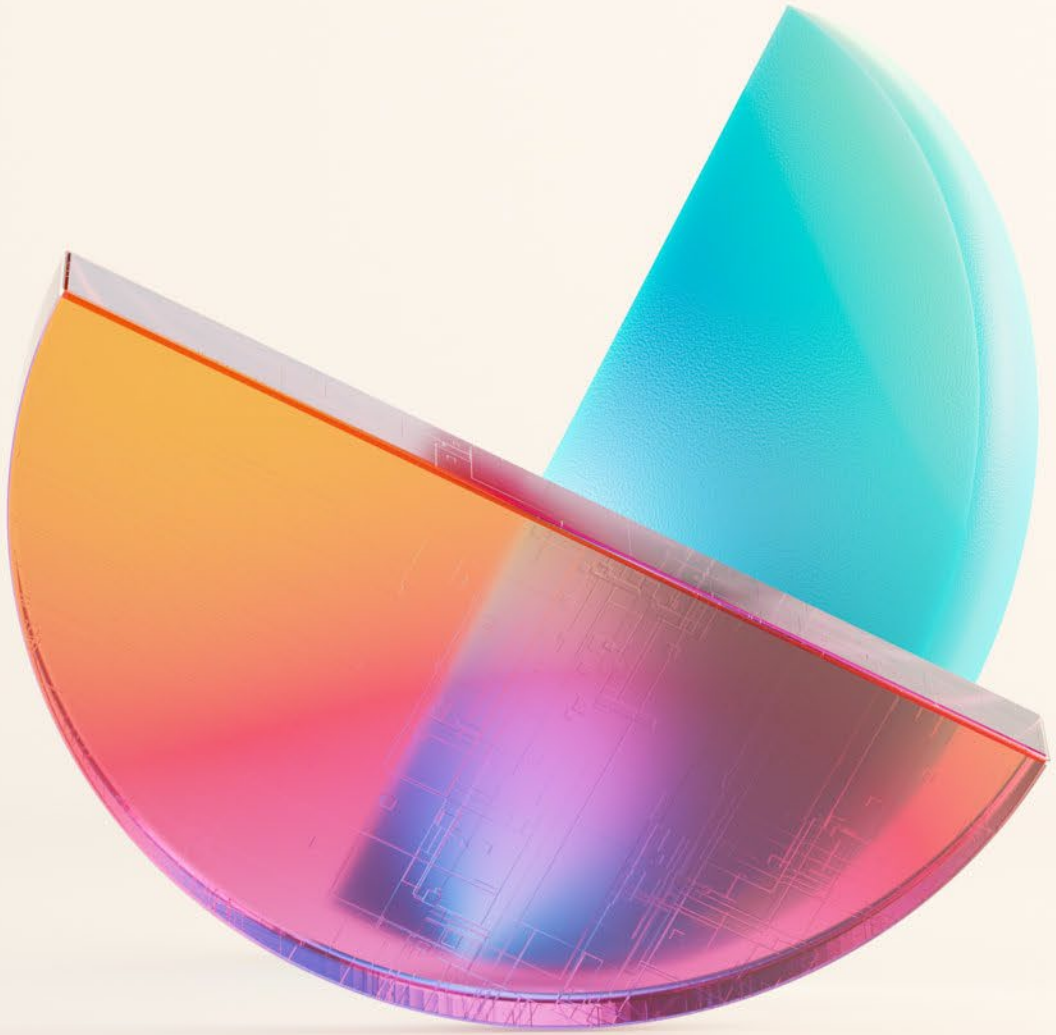
**Become a Microsoft Certified Fabric Data Engineer Associate – take Exam DP-700 for free!**

Visit the Fabric Community Lounge to learn more about this *limited-time offer*.

**aka.ms/fabcon/dp700**

Microsoft

Microsoft Fabric
Community Conference



# Learn more about Microsoft Fabric

Power your AI transformation with a complete data platform

Microsoft

# Get Involved in the Fabric Community



**aka.ms/FabricCommunity**
Connect with community members, ask questions, and learn more about Fabric



**aka.ms/FabricUserGroups**
Find a user group that matches your interests in your area or online



**aka.ms/SuperUsers**
Spread your Fabric knowledge, insights, and best practices with others




**aka.ms/MVP**
Technology experts that share their knowledge and passion with the community